Unit 2

Differences between Lists, Tuples, and Sets

Feature	List	Tuple	Set
Syntax	`[ ]`	`( )`	`{ }`
Ordered	Yes	Yes	❌ No
Indexed	Yes	Yes	❌ No
Mutable	✅ Yes (can change)	❌ No (immutable)	✅ Yes
Duplicates Allowed	Yes	Yes	❌ No
Data Types	Can be mixed	Can be mixed	Can be mixed
Speed	Moderate	Faster than list	Fast (for membership test)
Use Case	General-purpose data	Fixed data	Unique items
Memory Usage	More	Less	Moderate

Methods

Strings	Lists	Tuples	Sets	Dictionary
Case Conversion:	**Adding elements to lists:**	count()	**Adding elements to Sets:**	**Adding elements to dictionary:**
upper()	append()	index()	add()	update({dict2})
lower()	insert()		update(set2)	
title()	extend()	max()		
captalize()		min()		
Content checking:	**Deleting elements from lists:**	len()	**Deleting elements from Sets:**	**Deleting elements from Dictionary:**
islower()	pop()		pop()	pop()
isupper()	remove()		remove()	popitem()
isdigit()	clear()		discard()	clear()
isalnum()	del		clear()	del
isspace()			del	
find()	sort()		Max(), min(), len	Keys()
replace()	reverse()			values()
split()	Count()			items()
join()	index()			get()
len, count(), index()	max(), min(), len			

1-Mark Questions

Strings:

1. **Q:** How do you create a string in Python?
   A: By enclosing characters in single (`' '`), double (`" "`), or triple quotes (`''' '''` / `""" """`).

2. **Q:** What is string indexing?
   A: Accessing individual characters of a string using their position (index).

3. **Q:** What is the index of the first character in a Python string?
   A: `0` (zero-based indexing).

4. **Q:** Write the output of `"Hello"[1]`.
   A: `e`.

5. **Q:** What does slicing do in strings?
   A: It extracts a portion (substring) of a string using `[start:end]`.

6. **Q:** Write the output of `"Python"[::-1]`.
   A: `"nohtyP"` (reversed string).

7. **Q:** Name one string method used to convert string to uppercase.
   A: `upper()`.

8. **Q:** Which method is used to remove whitespace from both ends of a string?
   A: `strip()`.

9. **Q:** Is a Python string mutable or immutable?
   A: Immutable.

10. **Q:** Write the output of `"apple".find("p")`.
    A: `1`.

Lists:

1. **Q:** How do you create an empty list in Python?
   A: `my_list = []` or `my_list = list()`

2. **Q:** What is the index of the first element in a Python list?
   A: `0`

3. **Q:** Write the slicing expression to get the first 3 elements of a list `nums`.
   A: `nums[:3]`

4. **Q:** Which keyword is used in **list comprehension**?
   A: `for`

5. **Q:** Name one method used to add an element to a list.
   A: `append()`

Tuples:

6. **Q:** Can tuples be modified after creation?
   A: No, tuples are immutable.

7. **Q:** How do you access the second element of a tuple `t`?
   A: `t[1]`

8. **Q:** Name a method used with tuples.
   A: `count()` or `index()`

Sets:

9. **Q:** How do you create an empty set in Python?
   A: `set()` (not `{}` because that creates a dictionary)

10. **Q:** Which operation finds common elements between sets?
    A: Intersection (`&` or `.intersection()`)

11. **Q:** Name one method used with sets.
    A: `add()`, `remove()`, `union()`

Dictionaries:

12. **Q:** How do you create an empty dictionary in Python?
    A: `{}` or `dict()`

13. **Q:** How do you access the value of key `"name"` in dictionary `d`?
    A: `d["name"]`

14. **Q:** Name one method used with dictionaries to get all keys.
    A: `keys()`

15. **Q:** Write an example of dictionary comprehension to create a square dictionary for numbers 1 to 3.
    A: `{x: x**2 for x in range(1, 4)}`

16. **Q:** Which function is commonly used to iterate over key-value pairs in a dictionary?
    A: `.items()`

🔹 2-Mark Questions

Strings:

1. **Q:** Explain string immutability in Python with an example.
   A: Strings cannot be changed after creation.

2. **Q:** Differentiate between `isalpha()` and `isdigit()` methods.
   A: `isalpha()` → Returns `True` if all characters are alphabets.

- `isdigit()` → Returns `True` if all characters are digits.

3. **Q:** What is string formatting? Give an example.
   A: Inserting variables inside a string using `format()` or f-strings.

Lists:

1. **Q:** Differentiate between `append()` and `extend()` methods in lists.
   A:
   - `append()` adds a single element at the end of the list.
   - `extend()` adds multiple elements from another iterable to the list.
2. **Q:** Write a Python program to create a list of squares of numbers from 1 to 5 using list comprehension.
   A:

```
squares = [x**2 for x in range(1, 6)]
print(squares)   # [1, 4, 9, 16, 25]
```

Tuples:

3. **Q:** Why are tuples faster than lists in Python?
   A: Tuples are immutable and stored in a fixed way, so Python optimizes memory usage and lookup, making them faster than lists.
4. **Q:** Write a Python statement to find the index of element `20` in tuple `t = (10, 20, 30)`.
   A:

```
t = (10, 20, 30)
print(t.index(20))
```

Sets:

5. **Q:** Write the difference between `remove()` and `discard()` in sets.
   A:
   - `remove()` raises an error if the element is not present.
   - `discard()` does not raise an error if the element is missing.
6. **Q:** Write a Python program to find the union of two sets.
   A:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B)
```

Dictionaries:

7. **Q:** How is a dictionary different from a list in Python?
   A:
   - Dictionary stores data as **key-value pairs**, while a list stores elements sequentially by index.
   - Dictionaries are unordered (before Python 3.7) but lists are ordered.
8. **Q:** Write a dictionary comprehension to create a dictionary of numbers 1 to 5 with their cubes.
   A:

```
cubes = {x: x**3 for x in range(1, 6)}
print(cubes)
```

5-Mark Questions and Answers

Q1. Explain string slicing with an example. Write a program to reverse a string using slicing.

Answer: **String slicing** allows extracting substrings using `[start:end:step]`.

```
# Slicing examples:



text = "PythonProgramming"

print(text[0:6])    # Output: Python
print(text[-11:-6]) # Output: Progr

# Reverse string using slicing
reversed_text = text[::-1]
```

```
print("Reversed:", reversed_text)
```

 # Output: gnimmargorPnohtyP

****Q2. What are common string methods? Demonstrate at least four with examples.****

Answer: Common string methods:

1. `upper()` → converts to uppercase.
2. `replace()` → replaces substrings.
3. `split()` → splits a string into a list.
4. 'isupper()' → Checks whether the string is in uppercase or not.

Eg:

```
s = "hello world"
print(s.upper())        # HELLO WORLD
print(s.replace("world","Python"))    # hello Python
print(s.split())        # ['hello', 'world']
print(s.isupper())      # False
```

****Q3. Explain list comprehension with an example. Write a program to generate a list of squares of numbers from 1 to 10.****

Answer:
* **List comprehension** provides a concise way to create lists.
* **Syntax**: [expression for item in iterable if condition]

```
# List comprehension example
squares = [x**2 for x in range(1, 11)]
print("Squares:", squares)
```

Output: `[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]`

****Q4. Differentiate between lists and tuples. Write a program to access elements using indexing.****

Answer:
* **Lists**: Mutable (can be changed).
* **Tuples**: Immutable (cannot be changed).

```
# List and Tuple example
my_list = [10, 20, 30]
my_tuple = (100, 200, 300)

print("List element at index 1:", my_list[1])   # 20
print("Tuple element at index 2:", my_tuple[2]) # 300
```

****Q5. Explain set operations with an example. Write a program to find common elements between two sets.****

Answer: **Sets** support operations like **union, intersection, difference**
Eg:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print("Union:", A | B)          # {1,2,3,4,5,6}
print("Intersection:", A & B)   # {3,4}
print("Difference:", A - B)     # {1,2}
```

****Q6. What is dictionary comprehension? Write a program to create a dictionary of numbers and their squares.****

Answer:
* **Dictionary comprehension** is a compact way of creating dictionaries.
* **Syntax**: {key: value for item in iterable}

Eg:

```
squares_dict = {x: x**2 for x in range(1, 6)}
print("Squares Dictionary:", squares_dict)
```

Output: `{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}`

****Q7. Write a program to iterate over a dictionary and print keys and values. Explain three dictionary methods.****

Answer:

```
student = {"name": "Ravi", "age": 20, "marks": 88}

# Iterating
for key, value in student.items():
    print(key, ":", value)

# Methods
print(student.keys())   # dict_keys(['name','age','marks'])
print(student.values()) # dict_values(['Ravi',20,88])
print(student.items())  # dict_items({"name": "Ravi", "age": 20, "marks": 88})
```

* `keys()` → returns all keys.
* `values()` → returns all values.
* ‘items()’ → returns all keys and values

More 5-Mark Questions:

Strings:

1. **Write a Python program to check whether a given string is a palindrome.**

   A) **s = "malayalam"**
   if s == s[::-1]:
   print("It is a palindrome")
   else:
   print("it is not a palindrome")

2. **Write a Python program to check whether a string contains only digits.**

```python
s = input("Enter a string: ")

if s.isdigit():
    print("String contains only digits")
else:
    print("String has non-digit characters")
```

```
Enter a string: 344vk
String has non-digit characters
```

3. **Write a Python program to count the number of vowels and consonants in a string.**

```python
s = input("Enter a string: ").lower()
vowels = "aeiou"
v_count = 0
c_count = 0

for ch in s:
    if ch.isalpha():
        if ch in vowels:
            v_count += 1
        else:
            c_count += 1

print("Vowels:", v_count)
print("Consonants:", c_count)
```

```
Enter a string: Python is interesting and easy to learn
Vowels: 12
Consonants: 21
```

4. **Write a Python program to find the longest word in a given sentence.**

```python
s = input("Enter a sentence: ")
words = s.split()
longest = max(words, key=len)
print("Longest word:", longest)
```

```
Enter a sentence: Students are interested to learn Python
Longest word: interested
```

5. **Write a Python program to replace all spaces in a string with -.**

```python
text = input("Enter a string: ")
new_text = text.replace(" ", "-")
print("Modified string:", new_text)
```

```
Enter a string: This is Python Programming class
Modified string: This-is-Python-Programming-class
```

Lists:

1. **Write a Python program to read 5 numbers into a list and then print the second largest and second smallest numbers.**

```python
numbers = [12, 45, 67, 23, 89, 34]
numbers.sort()
print("Second largest number:", numbers[-2])
print("Second smallest number:", numbers[1])
```

```
Second largest number: 67
Second smallest number: 23
```

2. **Explain list slicing with suitable examples. Write a program to reverse a list using slicing.**

```python
my_list = [10, 20, 30, 40, 50]
print(my_list[2])       # 30 (indexing)
print(my_list[-1])      # 50 (negative index)
print(my_list[1:4])     # [20, 30, 40] (slicing)
print(my_list[::-1])    # Reverses the list
```

```
30
50
[20, 30, 40]
[50, 40, 30, 20, 10]
```

3. **Write a Python program using list comprehension to generate a list of even and odd numbers between 1 and 20.**

```python
# with comprehension
evens = [x for x in range(1, 21) if x % 2 == 0]
print("The even nos are", evens)
odds = [x for x in range(1, 21) if x % 2 != 0]
print("The odd nos are", odds)
```

```
The even nos are [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
The odd nos are [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

Tuples:

4. Explain any four properties of tuples with examples.

```
Properties of Tuples-

1. Tuples are Ordered.
2. Immutable → once created, elements cannot be changed.
3. Allows duplicates.
4. Defined using ( ).
5. Heterogeneous: Can store different data types
    (int, string, float, etc.).
6. Indexable and iterable.
```

5. **Write a Python program to create a tuple with numbers and print all the elements at odd indices**

```python
numbers = (10, 20, 30, 40, 50, 60, 70, 80)

print("Tuple:", numbers)
print("Elements at odd indices:")

# Accessing elements at odd indices using slic
for i in range(1, len(numbers), 2):   # start
    print(f"Index {i}: {numbers[i]}")
```

```
Tuple: (10, 20, 30, 40, 50, 60, 70, 80)
Elements at odd indices:
Index 1: 20
Index 3: 40
Index 5: 60
Index 7: 80
```

6. **Write a program to convert a list into a tuple and print its length.**

```python
# Creating a list
my_list = [5, 10, 15, 20, 25]

# Converting list to tuple
my_tuple = tuple(my_list)

# Printing tuple and its length
print("List:", my_list)
print("Tuple:", my_tuple)
print("Length of tuple:", len(my_tuple))
```

```
List: [5, 10, 15, 20, 25]
Tuple: (5, 10, 15, 20, 25)
Length of tuple: 5
```

Sets:

7. **Write a Python program to find common elements between two sets and also find elements present only in the first set.**

```python
# Create two sets
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7, 8}

# Finding common elements
common = set1.intersection(set2)
# Finding elements present only in the first set
only_in_first = set1.difference(set2)

# Displaying results
print("Common Elements:", common)
print("Elements only in Set 1:", only_in_first)
```

```
Common Elements: {4, 5}
Elements only in Set 1: {1, 2, 3}
```

8. **Explain any four methods of sets with examples.**

```python
# 1. update method - Add elements to a set
x = {"apple", "banana", "cherry"}
y = {"kiwi", "orange"}
x.update(y)
print(x)  # Output: {"apple", "banana", "cherry","kiwi","orange"}

# 2. clear() method - empties the set:
x = {"apple", "banana", "cherry"}
x.clear()   # Output: set()

# 3 del method - Deletes the set
x = {"apple", "banana", "cherry"}
del x

# 4 pop method - pops out any one element from the set
x = {"apple", "banana", "cherry"}
y = x.pop()
```

9. **Write a program to remove duplicate elements from a list using a set.**

```
1. Program to find unique elements from a list
```

```
numbers = [1, 2, 2, 3, 4, 4, 5, 6, 1, 7]
unique_numbers = set(numbers)   # removes duplicates
print("Unique elements:", unique_numbers)
```

```
Unique elements: {1, 2, 3, 4, 5, 6, 7}
```

Dictionaries:

10. Write a Python program to store names and marks of 3 students in a dictionary and print the student with the highest marks.

```
x = {"Alice": 85, "Bob": 92, "Charlie": 78}
highest = max(x.values())
topper = max(x, key=x.get)
print(f"Topper is: {topper} with {highest} marks")
```

```
Topper is: Bob with 92 marks
```

11. **Explain dictionary comprehension with an example. Write a program to create a dictionary of numbers and their squares from 1 to 10.**

```
squares = {x: x*x for x in range(1, 11) if x%2==0}
print(squares)
```

```
{2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

12. **Write a Python program to count the frequency of each character in a given string using a dictionary.**

```
text = "programming"
freq = {ch: text.count(ch) for ch in text}
print("Character Count:", freq)
```

```
Character Count: {'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1
, 'n': 1}
```